

Reporte de Práctica Semana 05: Monolito Mexi Banco

Martínez Pérez Diego Alexander

Sistemas Distribuidos

Fecha: 20 de septiembre de 2026

1. Levantamiento del Entorno con Docker Compose (Pasos 1 y 2)

Descripción

Se ejecutó la compilación del proyecto y la creación de los contenedores utilizando Docker Compose. Se corrigieron las configuraciones del Dockerfile y del script ejecutable mvnw para garantizar el despliegue correcto sobre Alpine Linux.

Verificación de Contenedores

A continuación se muestra el resultado del comando `docker compose ps`, confirmando que tanto la base de datos (db) como la aplicación (app) se encuentran en estado healthy:

```
✓Image mexi-banco-app          Built          221.2s
✓Network mexibanco             Created        0.5s
✓Volume mexi-banco_mexibanco_datos Created        0.1s
✓Container mexi-banco-db-1     Healthy       7.8s
✓Container mexi-banco-app-1    Started       7.8s
PS C:\Users\Diego Alexander\mexi-banco> docker compose ps
NAME                IMAGE              COMMAND                  SERVICE  CREATED      STATUS              PORTS
mexi-banco-app-1    mexi-banco-app    "java -jar app.jar"     app      8 hours ago  Up 8 hours (healthy)  0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
```

2. Pruebas de Endpoints en Postman (Pasos 3 y 4)

Descripción

Se importó la colección Mexi Banco en Postman y se ejecutaron las peticiones HTTP en el orden indicado para verificar el funcionamiento de los servicios del monolito: apertura de cuentas, transferencias, control de idempotencia (mismo SPEI repetido), estados de cuenta y manejo de errores.

Resultados de la Ejecución

The screenshot shows the Postman interface with a workspace named 'DiegoMtzPz's Workspace'. The active collection is 'Mexi Banco v05.1' and the selected request is 'Estado de cuenta del origen'. The request is a GET method with the URL '({baseUrl}) /movimientos?clabe= {{(origen)}}'. The response is a 200 OK status with a response time of 142 ms and a body size of 526 B. The response body is a JSON array with two objects:

```
1 [
2   {
3     "id": 5,
4     "clabeCuenta": "002180789957191616",
5     "tipo": "TRANSFERENCIA_ENVIADA",
6     "monto": -200.00,
7     "saldoResultante": 800.00,
8     "fecha": "2026-09-21T02:20:44.040197Z",
9     "referencia": "a 012180789957191616"
10  },
11  {
12    "id": 3,
13    "clabeCuenta": "002180789957191616",
```

The left sidebar shows a list of collections, including 'CLABE repetida (409)', 'Cuerpo sin clabe (400)', 'SPEI sin Idempotency-Key (400)', 'Saldo insuficiente (422)', 'Los errores no movieron el saldo (750)', 'Transferir a la misma cuenta (422)', 'Transferencia interna', 'Abrir cuenta destino (500)', 'Consultar destino (700)', 'Consultar origen (800)', 'Transferir 200 de origen a destino', 'Consultar la transferencia', 'Avisos del origen', 'Idempotencia', 'SPEI de 50 (primer envío)', 'SPEI de 50 (reintento, misma clabe)', and 'Consultar origen (750, no 700)'. The right sidebar shows a 'Learning to Use Postman' section with a 'Send my first request' button and a 'Describe what you need' text area.

Colección ejecutada: Mexi Banco

Pruebas integradas: Verificación automática de respuestas (código 200 OK, validaciones de negocio e idoneidad de respuestas).

The screenshot shows the Postman interface with a workspace named 'DiegoMtzPz's Workspace'. The active collection is 'Mexi Banco v05.1' and the selected request is 'Abrir cuenta origen (1000)'. The request is a POST method with the URL '({baseUrl}) /cuentas'. The response is a 201 Created status with a response time of 20.04 s and a body size of 230 B. The response body is a JSON object:

```
1 {
2   "id": 1,
3   "clabe": "002180789957191616",
4   "titular": "Ana",
5   "saldo": 1000
6 }
```

The left sidebar shows a list of collections, including 'Salud de la app', 'Consultar cuenta origen', 'Cuando algo sale mal', 'CLABE que no existe (404)', 'CLABE repetida (409)', 'Cuerpo sin clabe (400)', 'SPEI sin Idempotency-Key (400)', 'Saldo insuficiente (422)', 'Los errores no movieron el saldo (750)', 'Transferir a la misma cuenta (422)', 'Transferencia interna', 'Abrir cuenta destino (500)', 'Consultar destino (700)', 'Consultar origen (800)', 'Transferir 200 de origen a destino', 'Consultar la transferencia', 'Estado de cuenta del origen', 'ENVIRONMENTS', 'DOCUMENTS', 'SPECS', 'MOCKS', 'DATASETS', and 'FLOWS'. The right sidebar shows a 'Learning to Use Postman' section with a 'Send my first request' button and a 'Describe what you need' text area.

3. Análisis de la Arquitectura del Código (Paso 5)

Se inspeccionó la estructura del monolito en el directorio src/main/java/mx/mexibanco. El sistema está organizado por dominios de negocio y sigue una arquitectura en 4 capas principales:

1. Controller: Expone los endpoints REST para interactuar con los clientes.
2. Service: Contiene las reglas de negocio y lógica de las operaciones bancarias.
3. Repository: Maneja la persistencia y consulta de datos en PostgreSQL
4. Entity: Define las entidades del modelo relacional mapeadas en la base de datos.

Módulos Identificados

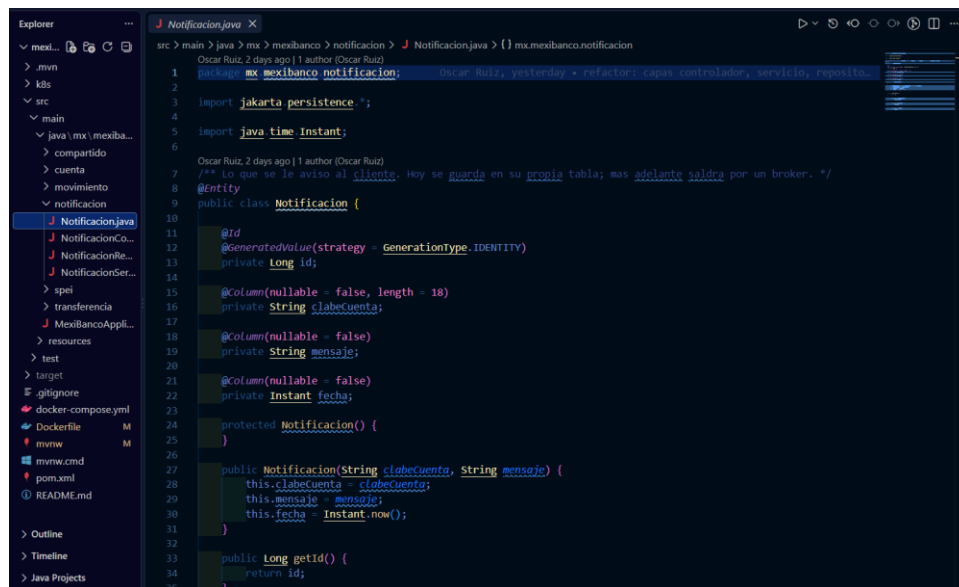
cuenta: Gestión de apertura y consulta de cuentas de usuario.

movimiento: Registro de entradas y salidas financieras.

transferencia: Lógica de transferencias internas entre cuentas.

notificacion: Sistema de avisos y alertas.

spei: Procesamiento e idempotencia de pagos interbancarios.



```
src > main > java > mx > mexibanco > notification > Notification.java > () mx.mexibanco.notification
Oscar Ruiz, 2 days ago | 1 author (Oscar Ruiz)
1 package mx.mexibanco.notification;
2
3 import jakarta.persistence.*;
4
5 import java.time.Instant;
6
7 /** Lo que se le aviso al cliente. Hoy se guarda en su propia tabla; mas adelante saldra por un broker. */
8 @Entity
9 public class Notification {
10
11     @Id
12     @GeneratedValue(strategy = GenerationType.IDENTITY)
13     private Long id;
14
15     @Column(nullable = false, length = 18)
16     private String clabeCuenta;
17
18     @Column(nullable = false)
19     private String mensaje;
20
21     @Column(nullable = false)
22     private Instant fecha;
23
24     protected Notification() {
25     }
26
27     public Notification(String clabeCuenta, String mensaje) {
28         this.clabeCuenta = clabeCuenta;
29         this.mensaje = mensaje;
30         this.fecha = Instant.now();
31     }
32
33     public Long getId() {
34         return id;
35     }
36 }
```

4. Análisis de Infraestructura Docker (Paso 6)

Observaciones sobre docker-compose.yml

Servicios: Define los contenedores db (PostgreSQL) y app (Spring Boot).

Healthchecks: Configura pruebas periódicas de salud para asegurar que la app no acepte peticiones hasta que la base de datos esté lista.

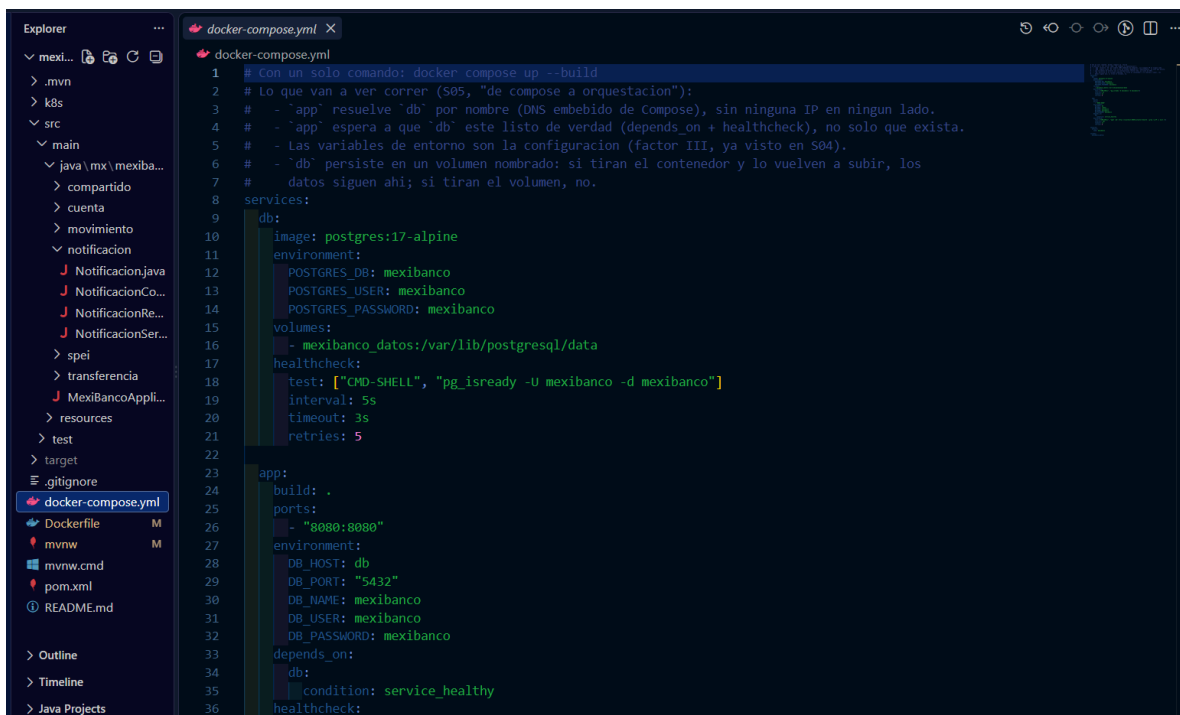
Redes y Volúmenes: Declara una red privada compartida entre servicios y un volumen para la persistencia de datos de PostgreSQL.

Observaciones sobre Dockerfile

Construcción Multietapa (Multi-stage build):

Etapas 1 (Build): Utiliza eclipse-temurin:21-jdk-alpine para compilar el proyecto y empaquetarlo con Maven.

Etapas 2 (Run): Utiliza eclipse-temurin:21-jre-alpine para copiar únicamente el archivo JAR resultante, reduciendo significativamente el tamaño final de la imagen y mejorando la seguridad en producción.



The screenshot shows an IDE with a project explorer on the left and a code editor on the right. The project explorer shows a project structure with folders like .mvn, k8s, src, main, java, mx, mexibanco, compartido, cuenta, movimiento, notificacion, and resources. The code editor displays the content of docker-compose.yml, which defines two services: db and app. The db service is based on postgres:17-alpine and includes environment variables for database configuration and a healthcheck. The app service is based on eclipse-temurin:21-jre-alpine and includes environment variables for database connection and a healthcheck that depends on the db service.

```
1 # Con un solo comando: docker compose up --build
2 # Lo que van a ver correr (S05, "de compose a orquestacion"):
3 # - 'app' resuelve 'db' por nombre (DNS embebido de Compose), sin ninguna IP en ningun lado.
4 # - 'app' espera a que 'db' este listo de verdad (depends_on + healthcheck), no solo que exista.
5 # - Las variables de entorno son la configuracion (factor III, ya visto en S04).
6 # - 'db' persiste en un volumen nombrado: si tiran el contenedor y lo vuelven a subir, los
7 #   datos siguen ahí; si tiran el volumen, no.
8 services:
9   db:
10     image: postgres:17-alpine
11     environment:
12       POSTGRES_DB: mexibanco
13       POSTGRES_USER: mexibanco
14       POSTGRES_PASSWORD: mexibanco
15     volumes:
16       - mexibanco_datos:/var/lib/postgresql/data
17     healthcheck:
18       test: ["CMD-SHELL", "pg_isready -U mexibanco -d mexibanco"]
19       interval: 5s
20       timeout: 3s
21       retries: 5
22
23   app:
24     build: .
25     ports:
26       - "8080:8080"
27     environment:
28       DB_HOST: db
29       DB_PORT: "5432"
30       DB_NAME: mexibanco
31       DB_USER: mexibanco
32       DB_PASSWORD: mexibanco
33     depends_on:
34       db:
35         condition: service_healthy
36     healthcheck:
```

5. Conclusión Personal

El monolito de Mexi Banco se encuentra desplegado correctamente y listo para su posterior modularización o migración a microservicios en las siguientes sesiones. Todas las pruebas funcionales de la API respondieron según lo esperado y la infraestructura en contenedores opera de manera estable. Sin embargo quiero destacar que al ser mi primera vez trabajando con algo de esta magnitud, no había tenido ese contacto con algo de este estilo, sin embargo, me pareció muy interesante y muy fácil de hacerlo aunque si me lleve un tiempo mas largo de lo prolongado para entender el correcto funcionamiento de todo.